Intelligent Document QA (RAG) - Comprehensive Project Documentation

Date: 2026-07-02
Audience: Technical and non-technical readers

Project Overview

Objective

Build an AI application that allows users to upload documents (PDF, DOCX, TXT), ask questions in natural language, and receive answers grounded in the uploaded content.

Business Value

- Reduces manual time spent reading long documents.
- Improves information accessibility for non-technical users.
- Provides traceable answers through section-based references.
- Supports both local demos and production-style deployment patterns.

Scope

- Document ingestion and parsing.
- Text chunking and embedding generation.
- Vector similarity retrieval.
- LLM-based answer generation with guardrails.
- API interface and Streamlit UI.

System Architecture

The system follows a modular Retrieval-Augmented Generation (RAG) architecture.

Syntax error in text

mermaid version 11.15.0

Architectural Design Principles

- Separation of concerns: each pipeline stage is isolated in its own module.
- Pluggability: backend and provider selection is configuration-driven.
- Defensive AI output handling: relevance and groundedness checks are built in.
- Progressive scalability: FAISS for local speed; pgvector for persistence.

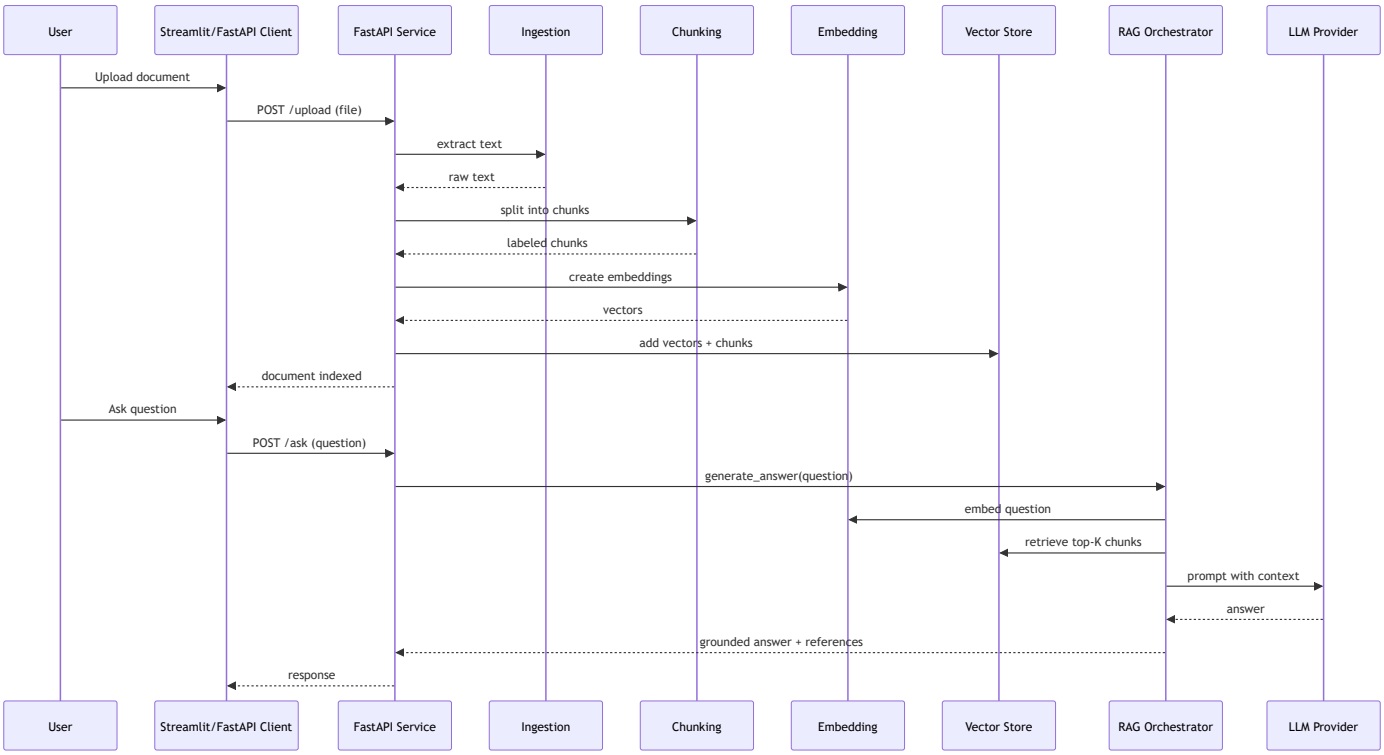
Technology Stack

Layer	Technologies	Purpose
Frontend	Streamlit	Document upload, Q&A interaction, session-based history
Backend API	FastAPI, Uvicorn, Pydantic	REST endpoints, validation, service runtime
Database / Vector Store	FAISS (in-memory), PostgreSQL + pgvector (persistent)	Similarity search over embeddings
AI / ML Frameworks	sentence-transformers, OpenAI SDK, Hugging Face Inference API	Embeddings + LLM answer generation
Data Processing	pdfplumber, python-docx, NumPy	Text extraction and vector handling
DevOps	Docker, Jenkins, pytest	Containerization, CI pipeline, testing
Third-party Integrations	OpenAI API, Hugging Face API	External inference services
Cloud Services	Optional (provider-hosted APIs for LLMs)	Managed model inference

Notes:

- No mandatory cloud platform dependency (AWS/Azure/GCP) is required to run the app.
- Cloud usage is optional through model provider APIs.

Application Workflow



Database Design

The app supports two retrieval modes:

1. FAISS in-memory index (default for local/demo speed).
2. PostgreSQL with pgvector (persistent storage and multi-run durability).

pgvector Table Design

Column	Type	Description
id	BIGSERIAL PK	Unique embedding row identifier
text	TEXT	Original chunk text
embedding	vector(384)	Dense vector from all-MiniLM-L6-v2
source_document	TEXT	Optional source filename/document id

Column	Type	Description
metadata	JSONB	Arbitrary metadata
created_at	TIMESTAMPTZ	Insertion timestamp

Indexes

- HNSW vector index on embedding for nearest-neighbor search.
- B-tree index on source_document for filtering/useful lookups.

API Overview

Endpoint	Method	Input	Output	Purpose
/upload	POST (multipart)	file (pdf/docx/txt)	{"message": "Document processed successfully"}	Build retrieval index from uploaded document
/ask	POST (JSON)	{"question": "..."}	{"answer": "..."}	Return answer grounded in indexed content

Request and Response Schemas

```
class QuestionRequest(BaseModel):
    question: str

class AnswerResponse(BaseModel):
    answer: str
```

- QuestionRequest input: natural-language user question.
- AnswerResponse output: generated answer text (optionally with references).

Key Functions and Code Explanations

This section uses focused snippets (not full source files) and explains each snippet in plain language.

1) API Upload Handler

```python
@app.post("/upload")
async def upload_document(file: UploadFile = File(...)):
    file_path = f"temp_{file.filename}"
    with open(file_path, "wb") as f:
        shutil.copyfileobj(file.file, f)

    text = extract_text(file_path)
    chunks = chunk_text(text)
    embeddings = embed_text(chunks)

    global vector_store
    vector_store = VectorStore(dim=len(embeddings[0]))
    vector_store.clear()
    vector_store.add(embeddings, chunks)

    os.remove(file_path)
    return {"message": "Document processed successfully"}
```

Purpose:

- Converts an uploaded file into a searchable vector index.

How it works:

- Saves the file temporarily, extracts text, splits text into chunks, embeds chunks, and stores vectors in the configured backend.

Inputs:

- Multipart file upload (PDF, DOCX, TXT).

Outputs:

- Success message indicating indexing completed.

Dependencies:

- extraction module, chunking module, embeddings module, vector store module.

Interaction with other components:

- Produces the index state consumed later by the /ask endpoint.

2) Text Chunking with Section Labels

```
def chunk_text(text: str) -> list[str]:
    if CHUNK_OVERLAP >= CHUNK_SIZE:
        raise ValueError("CHUNK_OVERLAP must be less than CHUNK_SIZE")

    chunks = []
    start = 0
    while start < len(text):
        end = start + CHUNK_SIZE
        chunk = text[start:end].strip()
        if chunk:
            label = _infer_section_label(chunk, len(chunks))
            chunks.append(f"[{label}] {chunk}")
        start = end - CHUNK_OVERLAP
    return chunks
```

Purpose:

- Splits long text into overlapping segments that preserve context and can be retrieved efficiently.

How it works:

- Uses fixed-size windows with overlap to avoid losing meaning at boundaries.
- Prefixes each chunk with a readable section label for traceable references.

Inputs:

- Raw extracted text.

Outputs:

- List of labeled chunks ready for embedding.

Dependencies:

- Configuration constants (CHUNK_SIZE, CHUNK_OVERLAP), section label helper.

Interaction with other components:

- Output feeds directly into embedding generation and later citation formatting.

3) Thread-safe Embedding Model Loader

```
_model = None
_model_lock = threading.Lock()

def get_model():
    global _model
    if _model is not None:
        return _model

    with _model_lock:
        if _model is None:
            from sentence_transformers import SentenceTransformer
            _model = SentenceTransformer("all-MiniLM-L6-v2")
    return _model
```

Purpose:

- Ensures the embedding model is initialized once and reused safely across requests.

How it works:

- Uses lazy loading and a lock to prevent concurrent duplicate model initialization.

Inputs:

- None directly (uses configured model name).

Outputs:

- Reusable SentenceTransformer instance.

Dependencies:

- sentence-transformers library and Python threading.

Interaction with other components:

- Called by embed_text for both document chunks and user questions.

4) Vector Backend Selection Wrapper

```
class VectorStore:
    def __init__(self, dim: int, backend: str | None = None):
        self.backend = (backend or config.VECTOR_DB_BACKEND).lower()
        if self.backend == "faiss":
            self._store = FaissVectorStore(dim)
        elif self.backend == "pgvector":
            self._store = PGVectorStore(dim)
        elif self.backend == "hybrid":
            self._store = HybridVectorStore([FaissVectorStore(dim), PGVectorStore(dim)])
        else:
            raise ValueError("Unsupported VECTOR_DB_BACKEND")
```

Purpose:

- Provides a single interface for multiple retrieval backends.

How it works:

- Reads runtime configuration and instantiates the selected backend strategy.

Inputs:

- Embedding dimension and optional backend override.

Outputs:

- Backend-specific store hidden behind a unified API.

Dependencies:

- FAISS implementation, pgvector implementation, hybrid strategy.

Interaction with other components:

- Called by upload flow to index data; queried by RAG flow during question answering.

5) RAG Orchestrator with Guardrails

```
def generate_answer(question, vector_store):
    query_embedding = embed_text([question])[0]
    context_chunks = _dedupe_chunks(vector_store.search(query_embedding, config.TOP_K))

    if not _has_relevant_context(question, context_chunks):
        return NO_RELEVANT_INFO_RESPONSE

    prompt = build_prompt(context_chunks, question)
    raw_answer = call_provider(prompt)
    return _finalize_answer(raw_answer, question, context_chunks)
```

Purpose:

- Orchestrates retrieval, prompt construction, LLM inference, and answer safety checks.

How it works:

- Embeds question, retrieves top-K chunks, validates relevance, queries model provider, validates groundedness, appends references.

Inputs:

- User question and active vector store.

Outputs:

- Final answer text, either grounded response or standardized no-relevant-info message.

Dependencies:

- embeddings helper, vector store, provider clients (OpenAI/Hugging Face), heuristic validators.

Interaction with other components:

- Invoked by /ask endpoint and returns user-facing answer content.

Role of Prompt Engineering

Prompt engineering is central to correctness and safety in this app.

Prompt Design Strategy

- Force context-bounded answering.
- Explicitly disallow external knowledge unless marked as external.
- Require fallback response when context does not support the question.
- Require references when section labels exist.

Prompt Template Pattern

```
Answer the question using ONLY the context below.
If unsupported, return exactly: "I couldn't find relevant information..."
Do not use outside knowledge.
Context:
{retrieved_context}

Question:
{question}
```

Context Management

- Top-K retrieval limits prompt length and focuses signal.
- Chunk overlap preserves semantic continuity.
- Deduplication removes repeated chunks before prompt assembly.

Prompt Optimization Techniques Used

- Deterministic generation (temperature = 0 for OpenAI path).
- Reference instructions to improve source traceability.
- Guardrail post-processing to reject ungrounded outputs.

Contribution to Performance and Accuracy

- Better grounding improves factual alignment.
- Context constraints reduce hallucination risk.
- Structured references improve user trust and verification.

LLM Integration

Model Selection

- OpenAI model (default): low-latency general-purpose generation.
- Hugging Face inference model: alternative provider pathway.
- Auto mode: race both providers and use first successful response.

Inference Flow

1. Convert question to embedding.
2. Retrieve top-K similar chunks.
3. Build grounded prompt.
4. Call provider.
5. Validate and normalize answer.

Embeddings

- Model: all-MiniLM-L6-v2.
- Vector dimension used by pgvector schema: 384.
- Embeddings generated for both document chunks and user queries.

Vector Database Usage

- FAISS for in-memory retrieval.
- pgvector for persistence and SQL-native vector search.
- Hybrid mode for mirrored writes and configurable primary search.

Retrieval Strategy

- Similarity search for candidate chunks.
- Deduplication of repeated retrievals.
- Heuristic relevance and groundedness validation before returning response.

AI Orchestration

- Centralized in a single generate_answer workflow.
- Includes provider fallback and graceful degradation behavior.

Security Considerations

Secret Management

- API keys are loaded from environment variables or .env for local use.
- Keys are not hard-coded in source.

Input and File Safety

- Uploads are restricted by supported file type handling logic.
- Files are stored temporarily and deleted after ingestion.

Data Protection

- In-memory mode minimizes persistence risk.
- pgvector mode enables controlled persistence with DB-level policies.

SQL and Injection Safety

- Identifier sanitization is applied for dynamic table names.
- Parameterized SQL is used for vector queries/inserts.

AI Safety Controls

- Context-only response instructions.
- Irrelevance and groundedness checks.
- Standardized fallback for unsupported answers.

Testing Strategy

Test Types

- Unit tests for ingestion, vector store helpers, and RAG logic.
- API tests for /upload and /ask endpoint behavior.
- Behavior tests for grounding and fallback rules.

Testing Approach

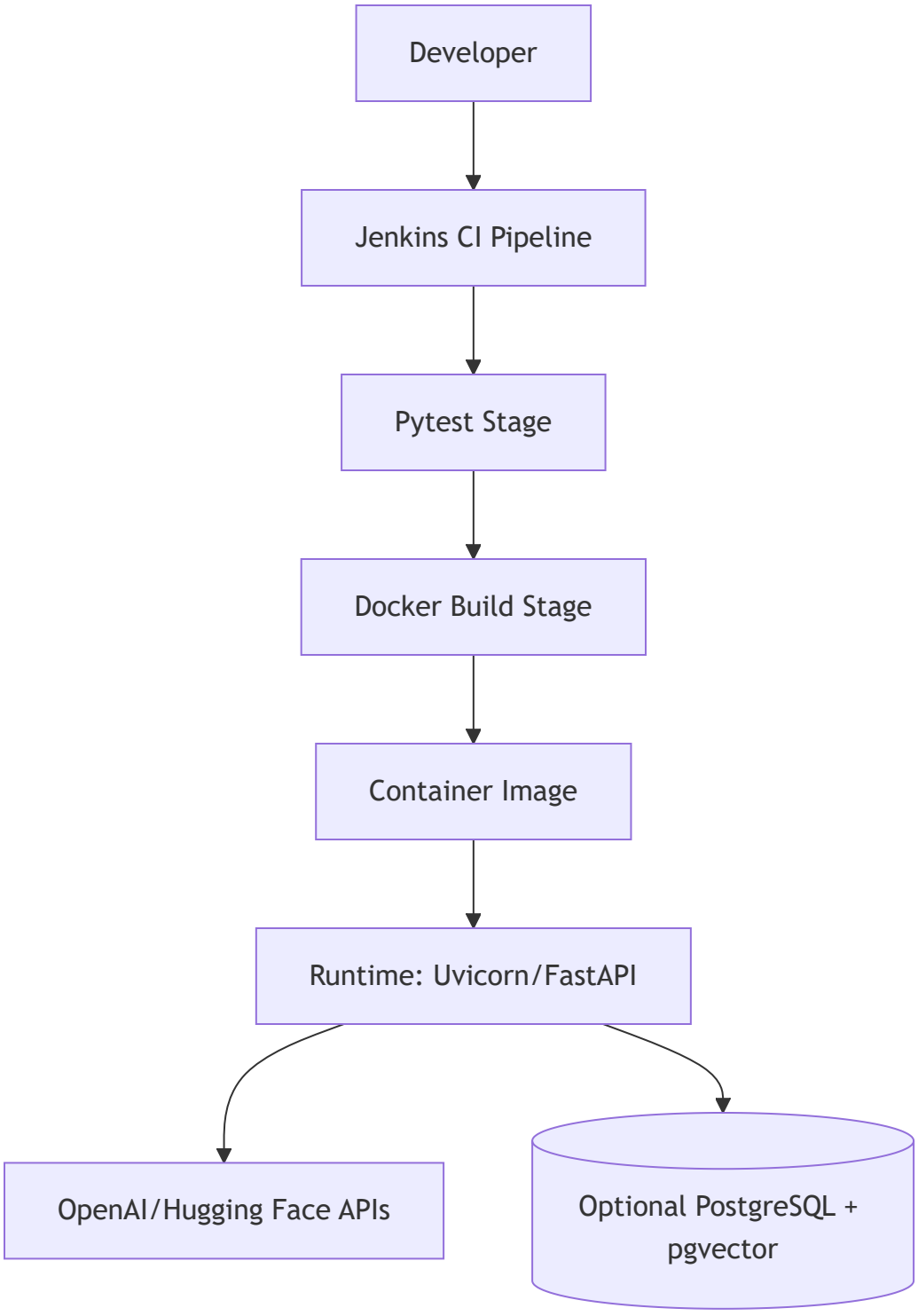
- Monkeypatching external dependencies to keep tests deterministic.

- Stubbing provider calls for fast and isolated validation.
- Coverage of edge cases: unsupported files, duplicate chunks, irrelevant context.

Quality Outcomes

- Faster CI runs with fewer flaky network-dependent failures.
- Confidence in both happy path and safety path behavior.

Deployment Architecture



Deployment Components

- Dockerfile packages app and dependencies.
- Jenkinsfile automates setup, test, and optional image build.
- Runtime can be local, VM, or container platform.

Operational Notes

- For production, prefer persistent backend and externalized configuration.
- Add centralized logging/metrics for retrieval and latency observability.

Future Enhancements

1. Multi-user document isolation (document IDs and tenant-aware retrieval).
2. Persistent metadata model with source attribution at chunk granularity.
3. Retrieval reranking (cross-encoder) to improve answer relevance.
4. Expanded API response schema (sources, confidence, latency, provider).
5. Observability dashboards for top-K quality, rejection reasons, and drift.
6. Security hardening (rate limits, authn/authz, file scanning).
7. Human-in-the-loop feedback loop for answer quality improvement.

Summary

This project is a strong implementation of a modular RAG system that balances usability, extensibility, and safety. It combines a clear API/UI experience with configurable retrieval infrastructure and practical AI guardrails. The design is suitable for education, internal knowledge assistants, and as a baseline for production-grade document intelligence platforms.